# Lecture 7
# Solutions

Homework 7: Review and merge

## Solutions to Homework 7

### Part A.

```
$ bash codeHints/L07/setup_hw7.sh
$ cd practice
$ git clone hw7_server.git hw7
$ cd hw7
$ git fetch
$ git log --oneline main..origin/teammate-feature     # 2 commits
$ git diff main...origin/teammate-feature             # "Helo, world" in greeting.txt
$ printf 'commits=2\ntypo-in=greeting.txt\nmerge-button=merge\n' > ../hw7_answers.txt
$ git switch teammate-feature
$ sed -i.bak 's/Helo/Hello/' greeting.txt && rm greeting.txt.bak    # or edit by hand
$ git commit -am "Fix typo in greeting"
$ git push
$ git switch main
$ git merge --no-ff --no-edit teammate-feature
$ git push
$ git branch -d teammate-feature
$ git push origin --delete teammate-feature
```

**7.2.** The branch adds two commits (`Add greeting` and `Document greeting`). The typo is `Helo` in `greeting.txt`. (`git diff` with three dots shows only what the branch added.)

**7.3.** `git switch teammate-feature` finds no local branch of that name, but there is `origin/teammate-feature`, so Git creates a tracking branch for you. A plain `git push` then updates the remote branch, as when you push a fix to an open pull request.

**7.4.** `--no-ff` forces a merge commit even though a fast-forward would be possible. This is what GitHub's button *Create a merge commit* does, hence the answer `merge`. (The checker expects a merge commit at the tip of the server's `main`; a fast-forward would fail.)

**7.5.** `git branch -d` works because the branch is merged. `git push origin --delete` removes it on the server (on GitHub, the *Delete branch* button).

**Part B.** Replace `A`, `B` by your account names.

**Round 1: shared repository (B opens the pull request).**

| Who | Step |
|---|---|
| A | Create a *public* repository `pr-practice` with a README. Settings → Collaborators → Add people: B. |
| B | Accept the invitation (e-mail, or `github.com/A/pr-practice/invitations`). |
| B | `git clone git@github.com:A/pr-practice.git practice/hw7_pr`, then `cd practice/hw7_pr`. |
| B | `git switch -c add-b-note`, create `notes-from-b.md`, `git add`, `git commit`, `git push -u origin add-b-note`. |
| B | On GitHub: *Compare & pull request*, title and description, *Create pull request*. |
| A | *Files changed*: inline comment with a suggestion; *Review changes* → *Request changes* → *Submit review*. |

| Who | Step |
|---|---|
| B | *Commit suggestion* (or fix locally: `git commit`, `git push`). |
| A | *Approve*; *Create a merge commit*; confirm. |
| B | *Delete branch*; locally `git switch main`, `git pull`, `git branch -d add-b-note`. |
| B | `printf 'my-pr=A/pr-practice#1\n' > ../hw7_pr_answers.txt` (from inside `hw7_pr`). |

**Round 2: fork (A opens the pull request).**

| Who | Step |
|---|---|
| B | Create a *public* repository `pr-practice-b` with a README. |
| A | *Fork* the repository, then `git clone git@github.com:A/pr-practice-b.git practice/hw7_pr` and `cd practice/hw7_pr`. |
| A | `git remote add upstream git@github.com:B/pr-practice-b.git`; `git remote -v` shows both remotes. |
| A | `git switch -c add-a-note`, add a file, commit, `git push -u origin add-a-note`. |
| A | On GitHub open the pull request; check that the base repository is `B/pr-practice-b`. |
| B | Review with a suggestion and *Request changes*. |
| A | *Commit suggestion*. |
| B | *Approve*; *Squash and merge*. |
| A | `git fetch upstream`, `git switch main`, `git merge upstream/main`, `git push origin main`. |
| A | `printf 'my-pr=B/pr-practice-b#1\n' > ../hw7_pr_answers.txt` (from inside `hw7_pr`). |

In Round 1, A has no local clone, and in Round 2, B has none; the names and the numbers `#1` above are placeholders. Each of you writes the pull request *you opened*: B the one from Round 1, A the one from Round 2. Both repositories must be public, because the checker asks GitHub's public service whether the pull request is merged. If the line is missing, the checker reports a failure; a private or non-existent pull request is a failure too, while an unreachable GitHub (no network, or more than 60 anonymous requests an hour) skips only that one check. The `upstream` check applies only to the fork route (A). After Round 2, A's `practice/hw7_pr` is the fork clone, so `origin` points at A's fork on github.com and `upstream` at B's repository.

**Common mistakes.**

- Merging with a fast-forward in 7.4 (`git merge teammate-feature` without `--no-ff`): the server's `main` then has no merge commit and the checker fails 7.4. Reset and redo.
- Opening the pull request *to your own fork*. Check that the base repository is the original owner's.
- B not accepting the collaborator invitation: the push in Round 1 fails with "Permission denied".
- Creating a repository as *Private*: the checker reports "GitHub cannot find ...".
- Forgetting `upstream`, then wondering why `git fetch upstream` fails ("not a git repository" or "no such remote").
- Writing the pull request number of your *partner's* request, or a private repository, in `hw7_pr_answers.txt`.
- `git branch -d` refusing after *Squash and merge*: expected (see the lecture); if the pull request really is merged, `git branch -D` is fine.